

Jetson Orin Nano 설정 가이드 (JetPack 설치 ~ 부팅 로고 커스터마이징)

대상 하드웨어: **Jetson Orin Nano Super Developer Kit** (P3767-0005 모듈 / P3768-0000 캐리어보드) 설치 버전: **JetPack 7.2 / L4T R39.2.0** (Ubuntu 24.04 기반) 호스트 PC: Ubuntu (x86_64) 최종 결과: 전원 ON 순간부터 데스크탑까지 **모든 로고를 회사 로고(LS Automotive)** 로 교체, 자동 로그인 + GUI 앱 자동 실행

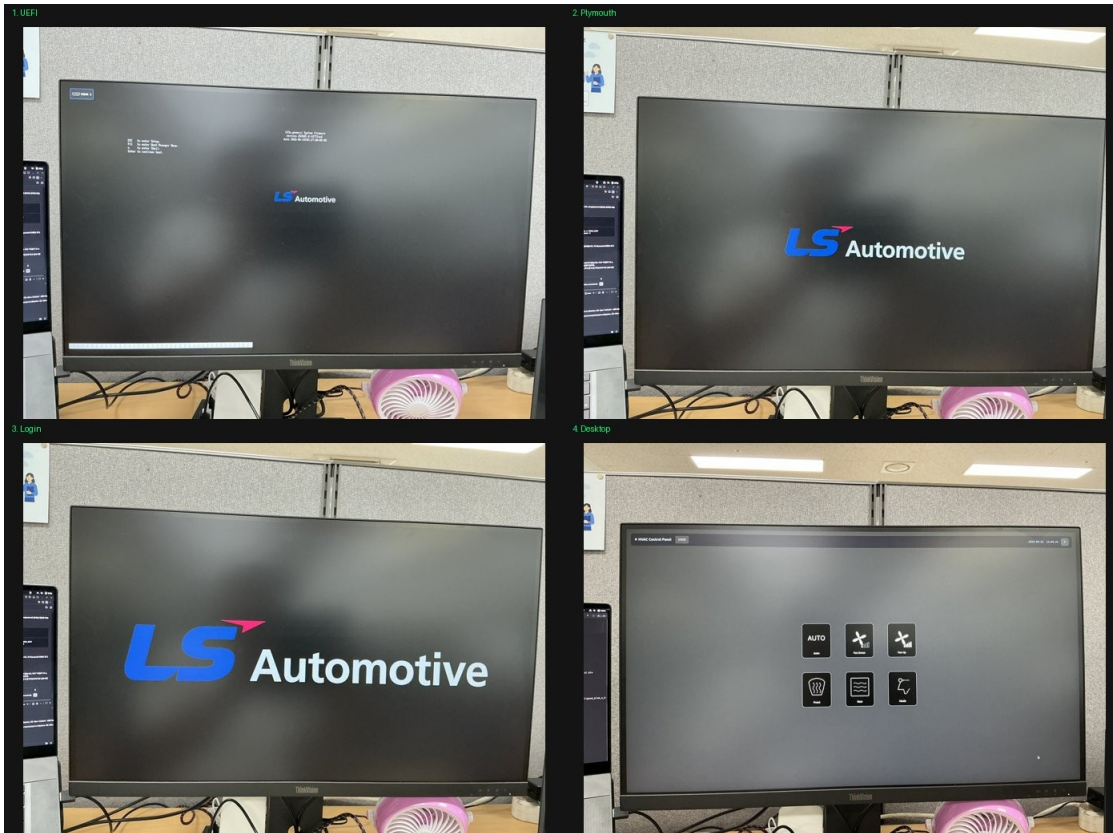


그림. 최종 완성 — 4단계 모두 회사 로고 (UEFI / Plymouth / 로그인 / 데스크탑)

0. 전체 그림 (로고가 나오는 4단계)

부팅하면 NVIDIA 로고가 서로 다른 4곳에서 나옵니다. 각각 출처가 달라서 수정 방법도 전부 다릅니다.

단계	보이는 시점	출처	수정 방법
1. UEFI 펌웨어	전원 ON 직후 (ESC to enter)	QSPI 펌웨어	UEFI 소스 빌드 후 QSPI

단계	보이는 시점	출처	수정 방법
어	Setup 화면)	내부 BMP	플래시
2. Plymouth	커널 부팅 중	rootfs의 Plymouth 테마	테마 교체 + initrd 재생성
3. 로그인 후 (~8초)	로그인 직후 데스크탑 뜨기 전	~/.xsessionrc 가 feh로 배경 표시	배경 PNG 교체
4. 데스크탑 배경	데스크탑	gsettings 배 경화면	gsettings 변경

1. JetPack 7.2 설치 (SDK Manager)

1-1. SDK Manager 설치 (호스트 PC)

1. developer.nvidia.com → “NVIDIA SDK Manager” 검색 → 로그인(NVIDIA 개발자 계정) → .deb 다운로드
2. 설치:

```
sudo apt install ~/Downloads/sdkmanager_*.deb
```

- o sudo: 관리자 권한(설치에 필요), 비밀번호 물어봄
- o apt install: 우분투 프로그램 설치 명령

1-2. Jetson을 강제 복구 모드(Force Recovery)로 진입

플래시하려면 보드가 “복구 모드”여야 합니다. 1. Jetson 전원 완전히 끄기 (전원 케이블 분리)
 2. USB-C 케이블로 Jetson ↔ 호스트 PC 연결 3. **REC 버튼을 누른 채로** 전원 케이블 연결 (또는 전원 버튼) 4. 2~3초 후 REC 버튼 떼기



그림. Jetson 보드 (하단부 - microSD 슬롯/버튼/커넥터)

확인 (호스트 PC 터미널):

lsusb | grep -i 0955

- 0955:7523 NVIDIA Corp. APX → 복구 모드 성공 ✓
- 0955:7020 ... L4T running on Tegra → 그냥 정상 부팅됨(실패), REC 타이밍 다시 시도

1-3. ⚠ SDK Manager GUI 크래시 → CLI로 우회 (중요 함정)

최신 호스트 OS에서 sdkmanager GUI가 즉시 죽는(SIGSEGV) 경우가 있습니다. 재설치/샌드박스 옵션 다 안 통할 때, **CLI 모드**로 우회합니다 (이게 핵심 노하우):

sdkmanager --query interactive --product Jetson

- ⚠ □□ | head 같은 파이프를 붙이지 말 것 — 파이프가 TTY를 없애서 방향키 메뉴가 깨지고 조용히 멈춥니다.
- 화면의 대화형 메뉴를 방향키 ↑ ↓ + Enter로 선택:
 - o Action: **Install**
 - o Product: **Jetson**
 - o System configuration: **Target Hardware**

- o Target hardware: Jetson Orin Nano modules
- o SDK version: **JetPack 7.2** (show all versions: Yes)
- o Install method: 모니터 있으면 **ISO Flash (Monitor)**, 없으면 **ISO Flash (Headless)**
- o Additional SDKs: 필요시 Holoscan 등 선택

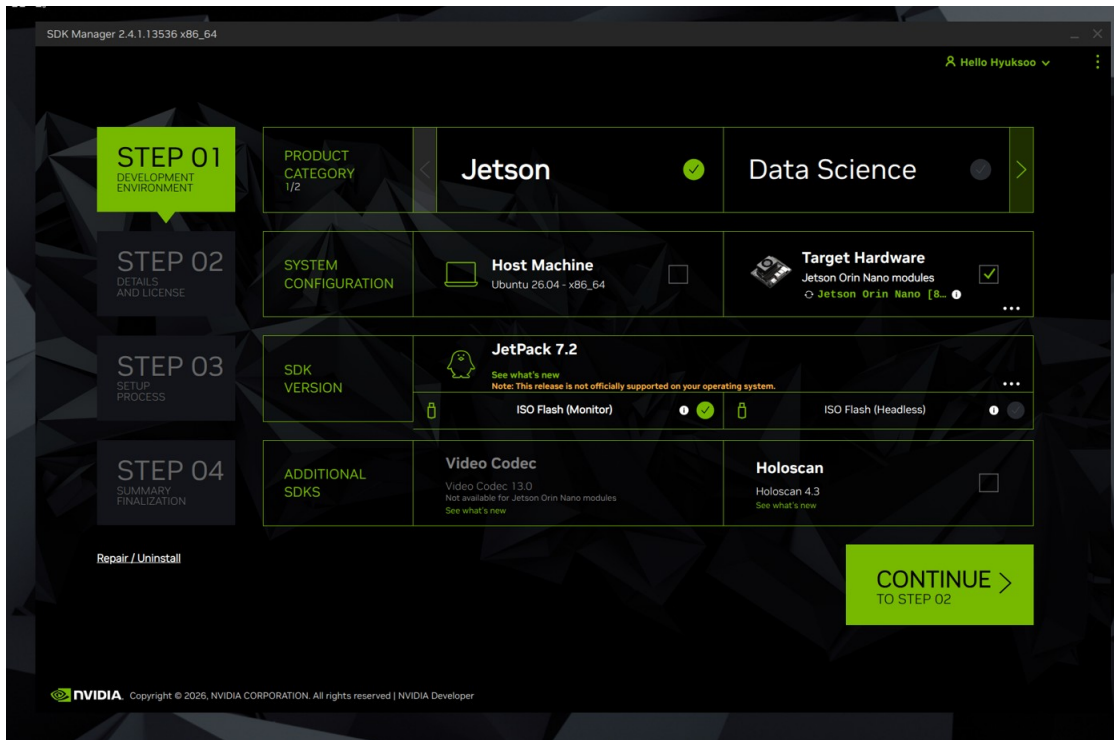


그림. SDK Manager STEP 01 — JetPack 7.2 / Target Hardware 선택

마지막에 아래 같은 실행 명령이 나옵니다(그대로 실행):

```
sdkmanager --action install --login-type devzone --product Jetson \
--target-os Linux --version 7.2 --show-all-versions \
--target JETSON_ORIN_NANO_TARGETS --install-method iso_flash \
--additional-sdk 'Holoscan 4.3'
```

- 참고: CLI의 --action install이 플래시 단계에 도달하면 GUI를 자기가 다시 띄우는데, 이 때는 GUI가 정상 작동합니다. (직접 GUI 실행할 때만 크래시 났음)

1-4. ISO 플래시 진행

- balenaEtcher로 ISO를 빈 **USB 메모리(8GB+)** 에 굽는 단계가 나옴 → USB 꽂고 진행
- Jetson에 그 USB 꽂고 부팅 → GRUB 메뉴에서 **"Install Jetson ISO 39.2.0"** 선택 → 자동 설치

- ⚠️ USB 빼지 말 것 — 일찍 빼면 EFI stub: Exiting boot services...에서 멈춤(블랙스크린). 전원 재투입으로 복구되지만 위험.

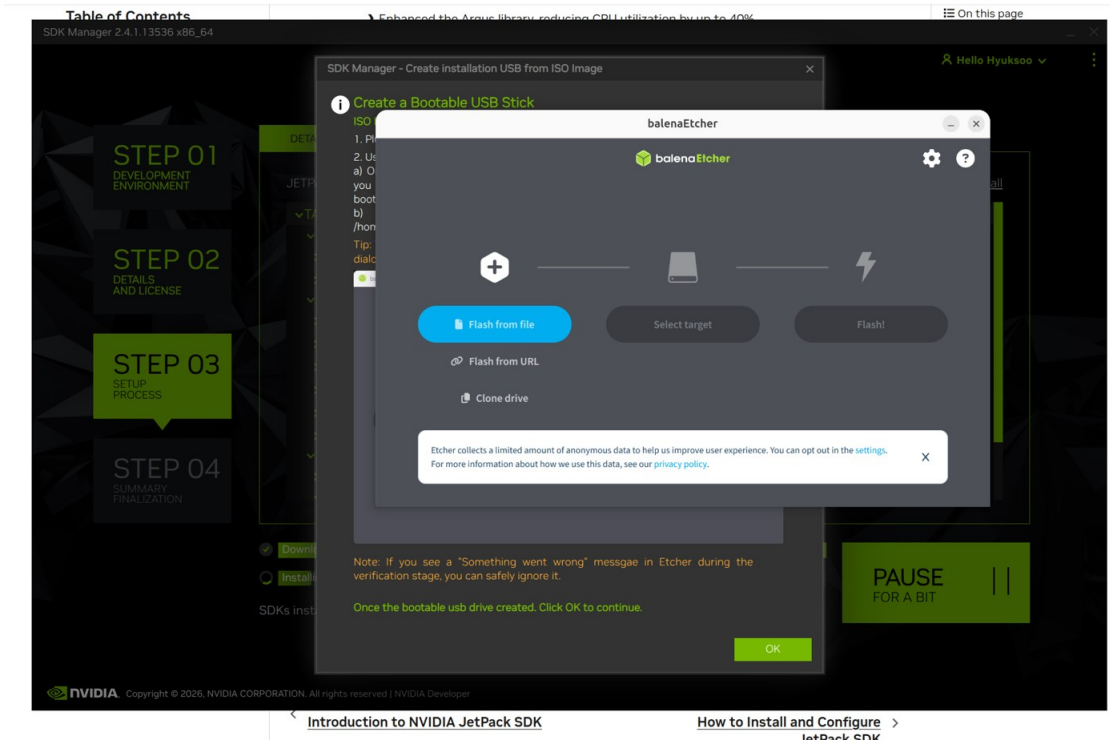


그림. balenaEtcher로 설치/USB [사진: Jetson 모니터의 GRUB 설치 메뉴 “Install Jetson ISO 39.2.0”]

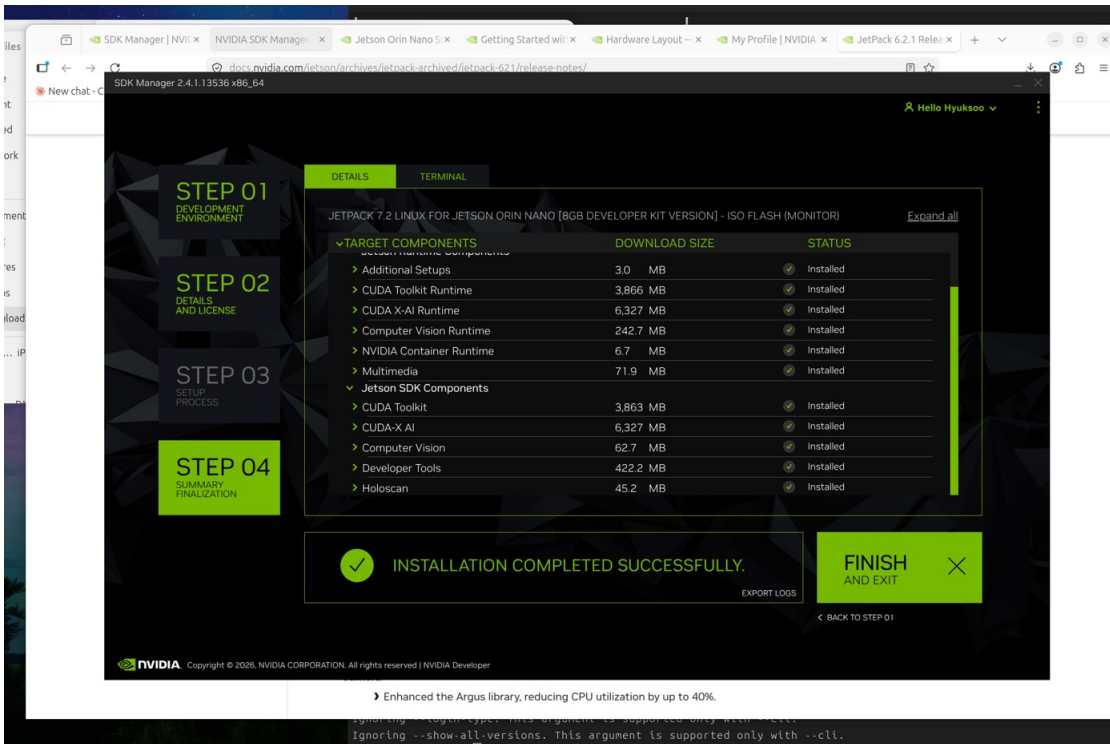


그림. SDK Manager STEP 04 — INSTALLATION COMPLETED SUCCESSFULLY

1-5. 계정 생성 + SDK 컴포넌트 설치

- 설치 후 Jetson이 재부팅 → 우분투 초기 설정 마법사 → **계정 생성** (예: hs / 0000)
 - “업데이트 확인” 프롬프트는 **Skip** (시간 절약)
 - 호스트 PC의 SDK Manager 창에서 “SDK components 설치” 대화상자 → 방금 만든 계정 정보 + USB 연결(IP 192.168.55.1) 입력 → CUDA/Holoscan 등 자동 설치
 - “INSTALLATION COMPLETED SUCCESSFULLY” 뜨면 완료
-

2. 호스트 PC에서 Jetson에 SSH 접속 (작업 편의)

SD카드를 매번 빼지 않고, USB 네트워크로 바로 작업할 수 있습니다.

연결 확인

```
ping -c 2 192.168.55.1
```

SSH 키 등록 (비밀번호 없이 접속되게, 최초 1회)

```
ssh-copy-id hs@192.168.55.1 # 비밀번호 0000 입력
```

접속

```
ssh hs@192.168.55.1
```

- 이후 호스트에서 `ssh hs@192.168.55.1` "명령어" 로 원격 실행 가능
 - 관리자 권한 명령은 `echo 0000 | sudo -S` 명령어 형태로 비밀번호를 파이프로 넣음
-

3. 로고 교체 ① — UEFI 펌웨어 (가장 복잡, 소스 빌드 필요)

전원 ON 직후 나오는 로고. QSPI 펌웨어 안에 BMP로 박혀있어 UEFI를 소스에서 빌드해야 합니다.

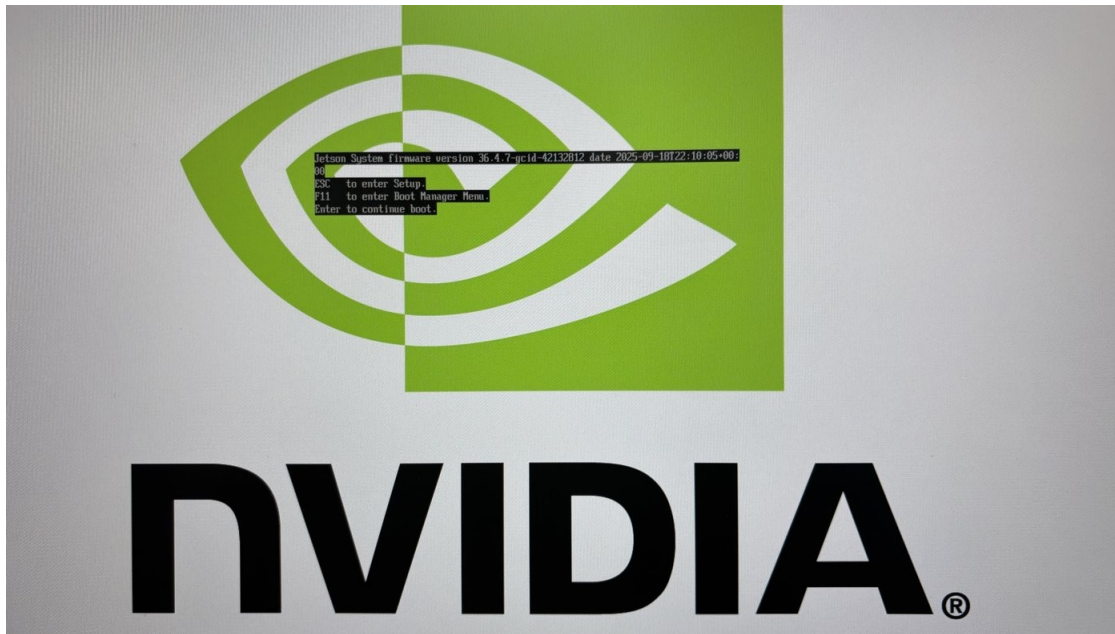


그림. (교체 전) UEFI 단계 — NVIDIA 로고 + 펌웨어 버전

3-1. Docker 빌드 환경

```
sudo apt install docker.io
```

```
sudo usermod -a -G docker $USER # 현재 계정을 docker 그룹에 추가  
# → 로그아웃/재부팅 후 적용 (안 되면 sudo docker로 실행)
```

3-2. UEFI 소스 워크스페이스 클론

```
sudo mkdir -p /build && sudo chown $USER:$USER /build
```

```
# git 사용자 정보 필수 (없으면 edkrepo clone 실패)
```

```
git config --global user.name "yourname"
```

```
git config --global user.email "you@example.com"
```

```
IMG=ghcr.io/tianocore/containers/ubuntu-22-dev:latest
```

```
# edkrepo 설정 초기화
```

```
sudo docker run --rm -v "$HOME":"$HOME" -e EDK2_DOCKER_USER_HOME="$HOME"  
$IMG init_edkrepo_conf
```

```
sudo docker run --rm -v "$HOME":"$HOME" -e EDK2_DOCKER_USER_HOME="$HOME"  
$IMG \
```

```
edkrepo manifest-repos add nvidia https://github.com/NVIDIA/edk2-edkrepo-  
manifest.git main nvidia
```

```
# 워크스페이스 클론 (~400k objects, 인터넷 끊기면 빈 폴더에서 재시도)
```

```
cd /build
```

```
sudo docker run --rm -w /build -v /build:/build -v "$HOME":"$HOME" \
-e EDK2_DOCKER_USER_HOME="$HOME" $IMG \
edkrepo clone nvidia-uefi NVIDIA-Platforms main
```

3-3. ⚠ 진짜 로고 파일 교체 (핵심 함정)

nvidiagray*.bmp (LogoMultipleGray) 가 아니라 **LogoSingleBlack** 이 실제 사용됩니다. -

확인: /build/nvidia-uefi/nvidia-config/t23x_general/.config 안

CONFIG_LOGO_IMPLEMENTER="Silicon/NVIDIA/Drivers/Logo/LogoSingleBlack.inf" - 교체
할 파일: Silicon/NVIDIA/Drivers/Logo/nvidiablack-1036x864.bmp (1036×864, 검은 배경)

회사 로고를 동일 규격(1036×864, 검은 배경, 24bit BMP)으로 만들어 덮어씁니다:

Python (PIL) 예시 — 회사 로고 PNG를 1036x864 검은 배경 BMP로

```
from PIL import Image
src = Image.open('회사로고.png').convert('RGBA')
w, h = 1036, 864
canvas = Image.new('RGB', (w, h), (0,0,0))
tw = int(w*0.5); s = tw/src.width; th = int(src.height*s)
logo = src.resize((tw, th), Image.LANCZOS)
canvas.paste(logo, ((w-tw)//2, (h-th)//2), logo)
canvas.save('/build/nvidia-uefi/edk2-nvidia/Silicon/NVIDIA/Drivers/Logo/
nvidiablack-1036x864.bmp', 'BMP')
```

3-4. UEFI 빌드

```
cd /build/nvidia-uefi
```

```
sudo docker run --rm -w /build/nvidia-uefi -v /build:/build -v "$HOME":"$HOME" \
-e EDK2_DOCKER_USER_HOME="$HOME" $IMG \
edk2-nvidia/Platform/NVIDIA/Tegra/build.sh \
--init-defconfig edk2-nvidia/Platform/NVIDIA/Tegra/DefConfigs/t23x_general.defconfig
```

- 결과물: /build/nvidia-uefi/images/uefi_t23x_general_DEBUG.bin (~3MB, UEFI 파티션 3.5MB 미만이어야 함)

3-5. QSPI에 플래시 (BSP 필요)

1. developer.nvidia.com → Jetson Linux 39.2 → **Driver Package (BSP)** 다운로드 (Jetson_Linux_R39.2.0_aarch64.tbz2, Sources 아님!)
2. 압축 해제 + 빌드한 바이너리로 교체:

```
tar xjf ~/Downloads/Jetson_Linux_R39.2.0_aarch64.tbz2 -C /tmp/
cp /build/nvidia-uefi/images/uefi_t23x_general_DEBUG.bin \
/tmp/Linux_for_Tegra/bootloader/uefi_bins/uefi_t23x_general.bin
sudo apt install -y libxml2-utils # flash.sh가 xmllint 필요
```


3. Jetson을 복구 모드로 진입 (1-2 참고)

4. ⚠ A/B 두 슬롯 모두 플래시 (A만 하면 B로 부팅돼서 효과 없음):

```
cd /tmp/Linux_for_Tegra
```

```
# A 슬롯
```

```
sudo ./flash.sh -k A_cpu-bootloader -c bootloader/generic/cfg/flash_t234_qspi.xml \
jetson-orin-nano-devkit-super mmcblk0p1
```

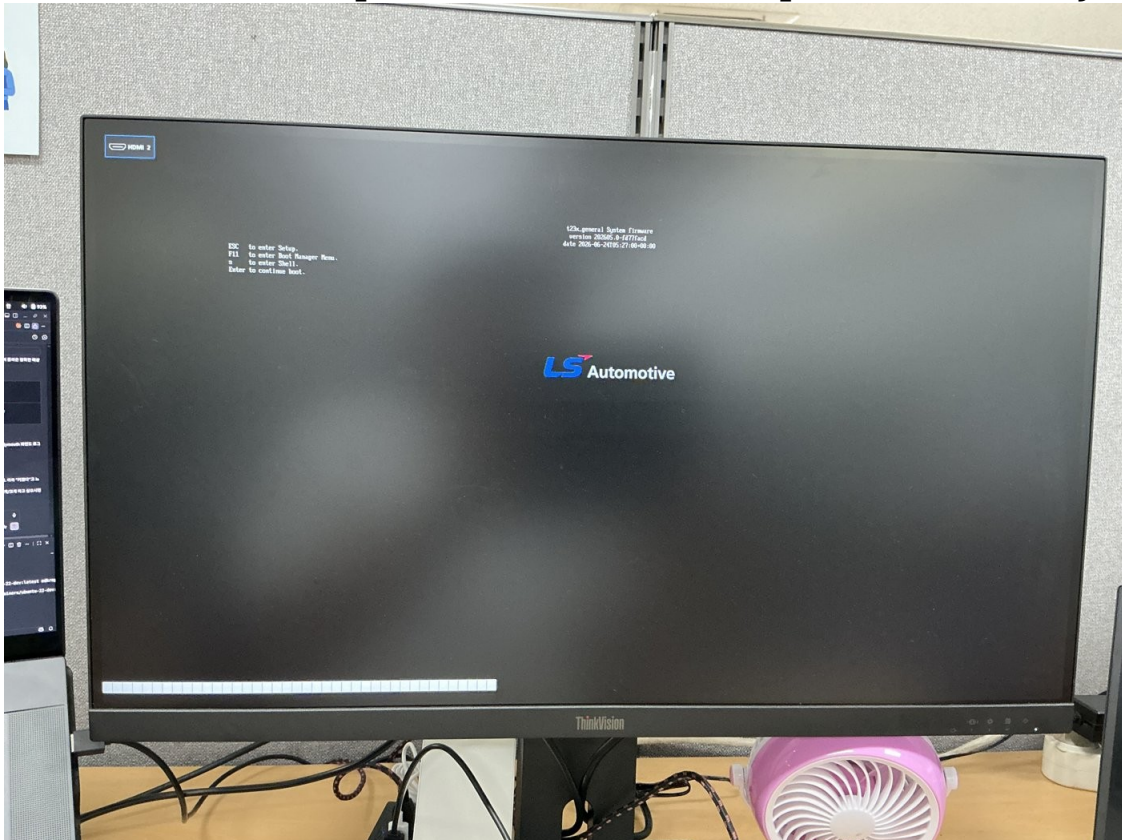
```
# → 다시 복구 모드 진입 후
```

```
# B 슬롯
```

```
sudo ./flash.sh -k B_cpu-bootloader -c bootloader/generic/cfg/flash_t234_qspi.xml \
jetson-orin-nano-devkit-super mmcblk0p1
```

o 이걸 파티션 일부만 쓰는 것 → SD카드의 OS는 안 지워짐

📷 [사진: flash.sh “The [A_cpu-bootloader] has been updated successfully” 출



력]

그림. (교체 후) UEFI 단계 — 회사 로고 표시

4. 로고 교체 ② — Plymouth 부팅 스플래시

커널 부팅 중 나오는 로고. (이하 모두 Jetson에서 직접 또는 SSH로 실행)

4-1. 커스텀 테마 생성

```
sudo mkdir -p /usr/share/plymouth/themes/company-logo
```

logo.png 준비 — 검은 배경, 약 480px 폭 (Plymouth 화면 실제 해상도가 약 1200px이라 480px ≈ 화면 40%):

```
from PIL import Image
src = Image.open('회사로고.png').convert('RGBA')
tw = 480; s = tw/src.width; th = int(src.height*s)
logo = src.resize((tw, th), Image.LANCZOS)
canvas = Image.new('RGB', (tw, th), (0,0,0))
canvas.paste(logo, (0,0), logo)
canvas.save('logo.png')
```

💡 Plymouth 내장 스케일러는 품질이 낮음 → **PIL(LANCZOS)**로 미리 정확한 크기로 줄이고, 스크립트에선 스케일 없이 **native** 배치해야 가장 선명함. 단, Plymouth 프레임버퍼가 1200px라 모니터가 2배 확대 → 네이티브 2560에서 그려지는 로그인 로고만큼 선명하게는 물리적으로 불가능.

company-logo.plymouth:

[Plymouth Theme]

Name=Company Logo

ModuleName=script

[script]

ImageDir=/usr/share/plymouth/themes/company-logo

ScriptFile=/usr/share/plymouth/themes/company-logo/company-logo.script

company-logo.script (스케일 없이 중앙 배치):

```
Window.SetBackgroundColor(0, 0, 0);
Window.SetBackgroundBottomColor(0, 0, 0);
logo.image = Image("logo.png");
logo.sprite = Sprite(logo.image);
logo.sprite.SetX(Window.GetWidth() / 2 - logo.image.GetWidth() / 2);
logo.sprite.SetY(Window.GetHeight() / 2 - logo.image.GetHeight() / 2);
fun refresh_callback() { }
Plymouth.SetRefreshFunction(refresh_callback);
```

4-2. 테마 활성화

```
sudo ln -sf /usr/share/plymouth/themes/company-logo/company-logo.plymouth \
/etc/alternatives/default.plymouth
echo -e "[Daemon]\nTheme=company-logo" | sudo tee /etc/plymouth/plymouthd.conf
```

4-3. ⚠ 부팅 옵션 + initrd 함정 (이게 핵심)

이 이미지는 기본 설정으로는 Plymouth가 안 뜸. 3가지를 고쳐야 함:

(a) 커널 부팅 옵션 — /boot/extlinux/extlinux.conf의 APPEND 줄에 splash quiet console=tty0 추가:

```
sudo cp /boot/extlinux/extlinux.conf /boot/extlinux/extlinux.conf.bak
# console=ttyTCU0,115200 뒤에 console=tty0 추가, video=efi:off 뒤에 splash quiet 추가
sudo sed -i 's/console=ttyTCU0,115200/console=ttyTCU0,115200 console=tty0/'
/boot/extlinux/extlinux.conf
sudo sed -i 's/video=efi:off/video=efi:off splash quiet/' /boot/extlinux/extlinux.conf
```

console=tty0이 없으면 Plymouth가 “화면 없음”으로 판단해 빈 텍스트 테마로 폴백 함 (필수!)

(b) --graphical-boot 옵션 — initramfs 스크립트와 systemd 유닛에 추가:

```
# initramfs 스크립트
sudo sed -i 's/--pid-file=/run/plymouth/pid#--pid-file=/run/plymouth/pid --graphical-boot#'\
/usr/share/initramfs-tools/scripts/init-premount/plymouth

# systemd 드롭인 (부팅/재시작/종료 화면용)
for svc in plymouth-start plymouth-reboot plymouth-poweroff; do
    sudo mkdir -p /etc/systemd/system/$svc.service.d
    mode=boot; [ "$svc" = plymouth-reboot ] && mode=reboot; [ "$svc" = plymouth-
poweroff ] && mode=shutdown
    printf '[Service]\nExecStart=\nExecStart=/usr/sbin/plymouthd --mode=%s --attach-to-
session --graphical-boot\n' "$mode" \
    | sudo tee /etc/systemd/system/$svc.service.d/override.conf
done
sudo systemctl daemon-reload
```

--graphical-boot이 없으면 콘솔이 VT가 아니라는 이유로 그래픽 스플래시를 거부함

(c) initrd □□□ — ⚠ update-initramfs -u는 “Nothing to do”로 실패함. 반드시 mkinitramfs -o 직접 사용:

```
sudo mkinitramfs -o /boot/initrd $(uname -r)
```

재부팅하면 Plymouth 회사 로고가 보임.



그림. Plymouth 부팅 스플래시 — 회사 로고

5. 로고 교체 ③ — 로그인 후 8초 로고 (.xsessionrc)

로그인 직후 데스크탑 뜨기 전 ~8초간 전체 화면 NVIDIA 로고. 원인은 ~/.xsessionrc:

이 줄이 feh로 NVIDIA 로고를 배경에 깔고 있음

`feh --no-fehbg --bg-scale /usr/share/backgrounds/NVIDIA_Login_Logo.png`

흰 배경 로고 = 이 파일. (검은 배경 UEFI 로고와 구분되는 결정적 단서였음)

해결: 그 PNG 파일을 회사 로고로 교체 (1920×1080, 통일감 위해 검은 배경 권장):

```
from PIL import Image
src = Image.open('회사로고.png').convert('RGBA')
w, h = 1920, 1080
canvas = Image.new('RGB', (w, h), (0,0,0)) # 검은 배경
tw = int(w*0.4); s = tw/src.width; th = int(src.height*s)
logo = src.resize((tw, th), Image.LANCZOS)
```



```
canvas.paste(logo, ((w-tw)//2, (h-th)//2), logo)  
canvas.save('NVIDIA_Login_Logo.png')
```

```
sudo cp /usr/share/backgrounds/NVIDIA_Login_Logo.png  
/usr/share/backgrounds/NVIDIA_Login_Logo.png.bak  
sudo cp NVIDIA_Login_Logo.png /usr/share/backgrounds/NVIDIA_Login_Logo.png
```

⚠ 화면 잔상 깜빡임(0.5초 미만)이 simpledrm→nvidia-drm 전환 시 보일 수 있는데, 이건 NVIDIA 비공개 드라이버 동작이라 수정 불가. 무시.



그림. 로그인 후 전환 화면 — 회사 로고



그림. 최종 데스크탑 — GUI 앱(HVAC 패널) 자동 실행

6. 마무리 설정 (데스크탑 배경 / 자동 로그인 / GUI 앱 자동실행)

6-1. 데스크탑 + 잠금화면 배경

```
sudo cp 회사배경.jpg /usr/share/backgrounds/company-bg.jpg
gsettings set org.gnome.desktop.background picture-uri
'file:///usr/share/backgrounds/company-bg.jpg'
gsettings set org.gnome.desktop.background picture-uri-dark
'file:///usr/share/backgrounds/company-bg.jpg'
gsettings set org.gnome.desktop.background picture-options 'zoom'
# NVIDIA 기본 배경 스크립트가 덮어쓰지 않게 마커 파일 생성
mkdir -p ~/.local/share/applications && touch
~/.local/share/applications/nvbackground_ubuntu
```

6-2. 자동 로그인 — /etc/gdm3/custom.conf

```
sudo sed -i 's/# AutomaticLoginEnable = true/AutomaticLoginEnable = true/'  
/etc/gdm3/custom.conf  
sudo sed -i 's/# AutomaticLogin = user1/AutomaticLogin = hs/' /etc/gdm3/custom.conf
```

6-3. GUI 앱 자동 실행 — ~/.config/autostart/

```
mkdir -p ~/.config/autostart  
cat > ~/.config/autostart/touch_panel.desktop <<'EOF'  
[Desktop Entry]  
Type=Application  
Name=Jetson Touch Panel  
Exec=/home/hs/jetson_touch_c/touch_panel  
Path=/home/hs/jetson_touch_c  
X-GNOME-Autostart-enabled=true  
EOF
```

💡 GPIO/SPI 쓰는 앱은 계정이 gpio 그룹에 있으면 sudo 없이 실행됨 (groups로 확인). root로 띄운 잔여 프로세스가 GPIO 라인을 잡고 있으면 [GPIO] line N 출력 요청 실패 에러 → sudo pkill -9 앱이름으로 정리.

7. SD카드 복제 (동일 보드 양산용)

지금 설정한 SD카드를 그대로 복사해서 같은 모델 보드에 쓰면 됩니다. - dd 또는 balenaEtcher로 전체 이미지를 떼서 새 SD카드(같거나 더 큰 용량)에 구우면 동일하게 동작 -
⚠️ 여러 보드를 같은 네트워크에 **동시 연결**하면 hostname/SSH 키가 같아 충돌 → 각 보드에
서 sudo hostnamectl set-hostname 새이름 으로 변경 - UEFI(QSPI) 펌웨어는 SD카드가 아니라 보드에 있으므로, **새 보드마다 3장(UEFI 플래시)을 따로 해줘야** UEFI 로고도 바뀜. (SD카드 복제로는 Plymouth/로그인/데스크탑 로고만 복제됨)

8. 핵심 함정 요약 (시간 잡아먹은 것들)

함정	해결
SDK Manager GUI 즉시 크래시	CLI 모드(--query interactive)로 우회, 파이프 금지
UEFI 로고 안 바뀜	LogoMultipleGray 아님, LogoSingleBlack/nvidiablack-1036x864.bmp 가 진짜
플래시했는데 로고 그대로	A/B 두 슬롯 모두 플래시 필요

함정	해결
Plymouth 안 뜸	console=tty0 + --graphical-boot 둘 다 필요
initrd 갱신 안 됨	update-initramfs -u 말고 mkinitramfs -o
로그인 후 NVIDIA 로고	GDM/확장 아님, ~/.xsessionrc의 feh
Plymouth 로고 흐림/크기	PIL로 미리 480px 축소 + native 배치 (Plymouth 스케일러 회피)

작성: 2026-06-24 / JetPack 7.2 (L4T R39.2.0) / Jetson Orin Nano Super Devkit